CNN CIA Test 1

Instructions

Marks: 15

- Identify and correct the errors without completely rewriting the program.
- Execute the corrected program successfully.

In [8]:
```
import numpy as np
import tensorflow as tf
import matplotlib.pyplot as plt

# ============================================================
# CNN DEBUGGING TEST
# ============================================================

# Create one local 28 x 28 grayscale image
image = np.zeros((28, 28, 1), dtype=np.float32)

image[5:8, 6:22] = 1.0
image[8:11, 18:22] = 1.0
image[11:14, 15:19] = 1.0
image[14:17, 12:16] = 1.0
image[17:20, 9:13] = 1.0
image[20:23, 6:10] = 1.0

# Display the input image
plt.figure(figsize=(4, 4))
plt.imshow(image, cmap="gray")
plt.title("Single Input Image")
plt.axis("off")
plt.show()

# Normalize the image
image = image.reshape(1, 28, 28, 1)

input_shape = (28, 28, 1)

print("Input shape:", image.shape)
print("Minimum:", image.min())
print("Maximum:", image.max())

# ============================================================
# CNN MODEL
# ============================================================

model = tf.keras.Sequential([
    tf.keras.Input(shape=(28, 28, 1)),
    tf.keras.layers.Conv2D(
        filters=32,
        kernel_size=(5, 5),
        activation="relu"
    ),
    tf.keras.layers.MaxPooling2D(
        pool_size=(3, 3),
```

```
        strides=(3, 3)
    ),
    tf.keras.layers.Conv2D(
        filters=8,
        kernel_size=(3, 3),
        activation="relu"
    ),
    tf.keras.layers.MaxPooling2D(
        pool_size=(2, 2)
    ),
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(
        32,
        activation="relu"
    )
])

model.summary()

# ============================================================
# FORWARD PASS
# ============================================================

output = model.predict(image, verbose=0)

print("\nFinal feature representation shape:", output.shape)
print("Forward pass completed successfully.")

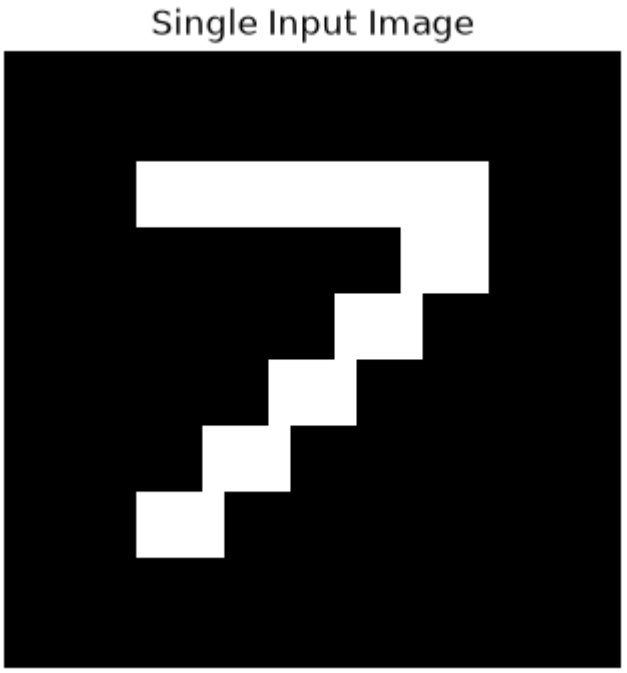
# ============================================================
# FIRST CONVOLUTION FEATURE MAP
# ============================================================

feature_model = tf.keras.Model(
    inputs=model.inputs[0],
    outputs=model.layers[0].output
)

feature_maps = feature_model(image, training=False)

print("First convolution output shape:", feature_maps.shape)

plt.figure(figsize=(5, 4))
plt.imshow(feature_maps[0, :, :, 0], cmap="gray")
plt.title("First Convolution Feature Map")
plt.axis("off")
plt.show()
```




```
Input shape: (1, 28, 28, 1)
Minimum: 0.0
Maximum: 1.0
```

Model: "sequential_4"

Layer (type)	Output Shape	Param #
conv2d_8 (Conv2D)	(None, 24, 24, 32)	832
max_pooling2d_8 (MaxPooling2D)	(None, 8, 8, 32)	0
conv2d_9 (Conv2D)	(None, 6, 6, 8)	2,312
max_pooling2d_9 (MaxPooling2D)	(None, 3, 3, 8)	0
flatten_4 (Flatten)	(None, 72)	0
dense_4 (Dense)	(None, 32)	2,336

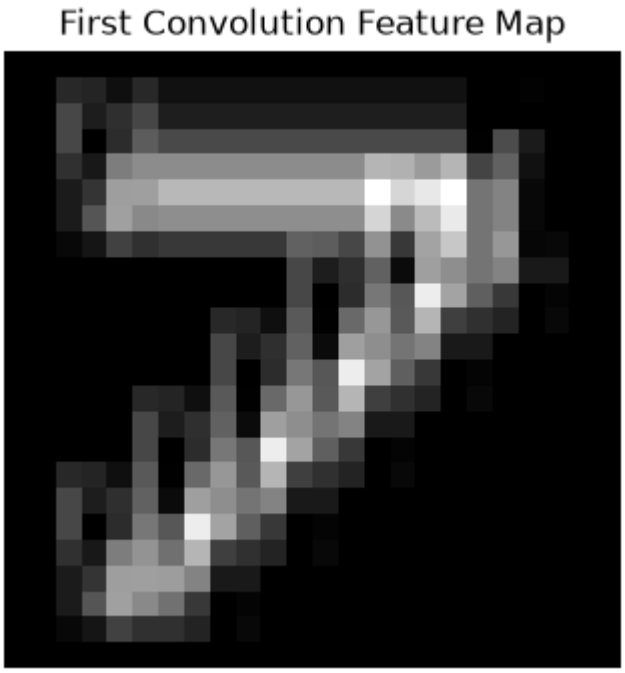
Total params: 5,480 (21.41 KB)

Trainable params: 5,480 (21.41 KB)

Non-trainable params: 0 (0.00 B)

```
WARNING:tensorflow:5 out of the last 5 calls to <function TensorFlowTrainer.make_
predict_function.<locals>.one_step_on_data_distributed at 0x7f4795bb2a20> trigger
ed tf.function retracing. Tracing is expensive and the excessive number of tracin
gs could be due to (1) creating @tf.function repeatedly in a loop, (2) passing te
nsors with different shapes, (3) passing Python objects instead of tensors. For
(1), please define your @tf.function outside of the loop. For (2), @tf.function h
as reduce_retracing=True option that can avoid unnecessary retracing. For (3), pl
ease refer to https://www.tensorflow.org/guide/function#controlling_retracing and
https://www.tensorflow.org/api_docs/python/tf/function for  more details.

Final feature representation shape: (1, 32)
Forward pass completed successfully.
First convolution output shape: (1, 24, 24, 32)
```



In [ ]: